REINFORCEMENT LEARNING

TinyPPO: Analysis of PPO-RLHF on TinyStories

Cristiano Baldassi · Scientific & Data Intensive Computing · April 2026

PART 01

Reinforcement Learning from Human Feedback

Motivation · Pipeline · KL Regularization

Why RLHF? | Three-Stage Pipeline | Reward Hacking | KL Penalty

01

Why RLHF?

PROMPT

"I had a really rough day at work today..."

1. PRETRAINED BASE

"...I had a rough day at work. Reasons include stress, deadlines, and a long commute home."

Pattern completion: continues the narrative.

2. INSTRUCT (SFT)

"I am sorry to hear that. Stress is difficult. Here are three tips to help you relax this evening."

Instruction following: acts as an assistant.

3. ALIGNED (RLHF)

"I'm so sorry. That sounds really draining. Do you want to vent about it, or would you prefer a distraction?"

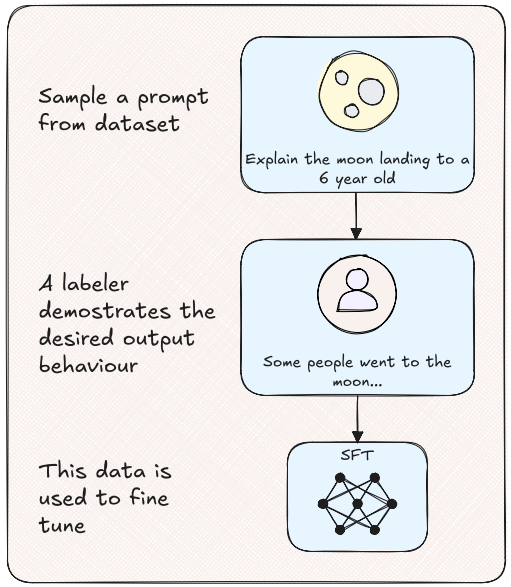
Human Preference: prioritizes empathy and nuance.

- **Pretraining learns language, not behavior.** Next-token prediction captures *how people write*, not *how the model should respond*.
- **SFT only goes so far.** Demonstrations show one good answer; they can't say *why* it's better than another.
- **Preferences resist gradient descent.** "This reply feels better" has no closed-form loss — preferences are comparative and non-differentiable.
- **RLHF closes the loop.** A reward model learns preferences as a scalar; RL optimizes the policy against it, with KL keeping the language grounded.

The Three-Stage Pipeline

01

Supervised Fine-Tuning

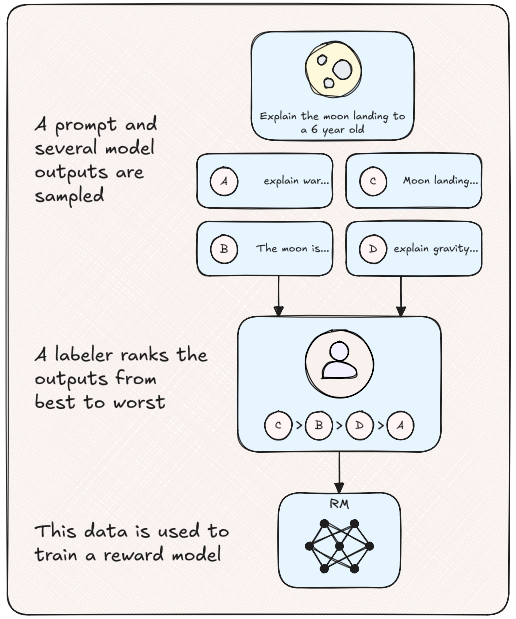


Fine-tune the base LM on curated demonstrations, yielding the reference policy π_{SFT}.

→

02

Reward Model

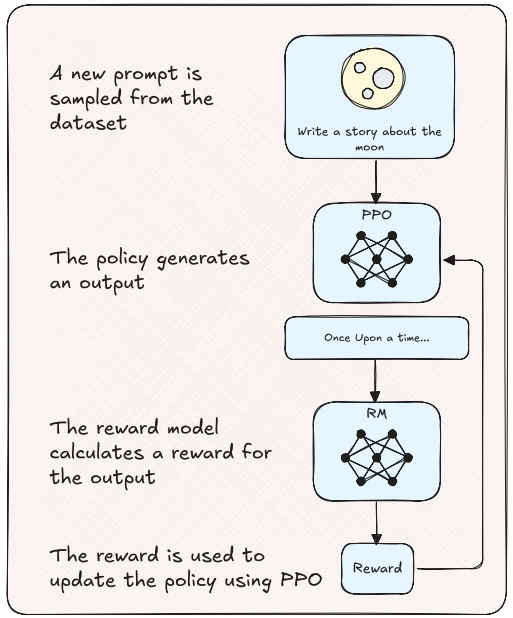


Collect human preference pairs and fit a reward model $r(x, y)$ that reproduces them as a scalar.

→

03

RL Fine-Tuning



Optimize π_θ against $r(x, y)$ with PPO, while a KL penalty keeps it tethered to π_{SFT}.

Reward Hacking & the KL Penalty

WITHOUT CONSTRAINT

- The policy quickly finds **blind spots** in the reward model and exploits them
- Degenerate outputs **score high** while saying nothing meaningful
- Linguistic coherence collapses *as reward is maximized*

SOLUTION

THE OBJECTIVE FUNCTION

$$R(x, y) = r(x, y) - \beta \mathbb{D}_{\mathrm{KL}}$$

where $\mathbb{D}_{\mathrm{KL}} = \ln \frac{\pi_\theta(y|x)}{\pi_{\mathrm{ref}}(y|x)}$

NO KL CONSTRAINT — Joy: 0.99

"I am so happy happy happy happy joyful great amazing amazing amazing amazing!!"

Repetitive gibberish; maximizes token-level sentiment.

WITH KL PENALTY — Joy: 0.85

"I found a new friend at the park today and we spent the afternoon playing together."

Maintains language structure while expressing joy.

PART 02

RLHF as an MDP

States · Actions · Rewards · Episodes

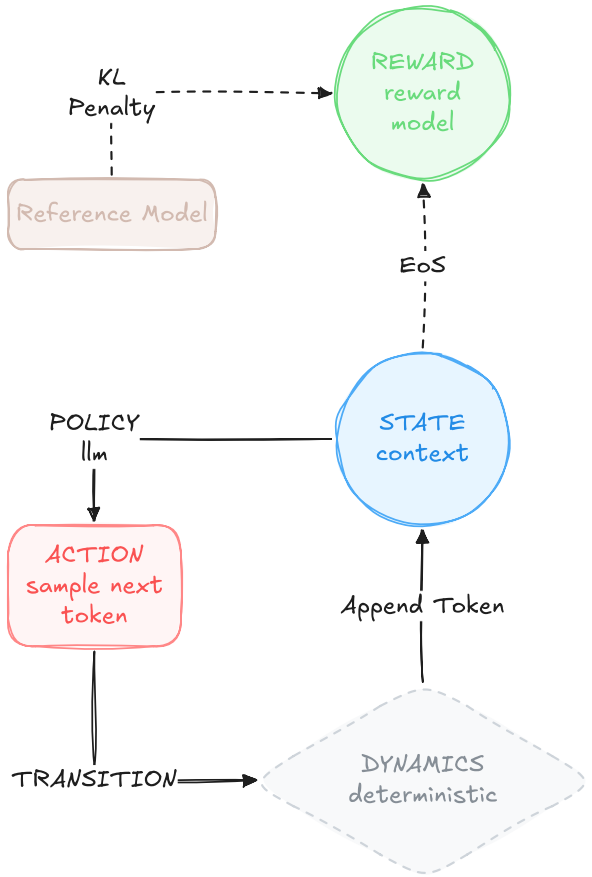
Language Gen as MDP | Per-Token Reward | What Makes It Unusual

02

Language Generation as an MDP

State s_t	**Context:** The token sequence so far (Prompt + partial response)
Action a_t	**Token:** The next sampled unit from the 50k+ vocabulary
Transitions P	**Deterministic:** Simply appending a_t to the current sequence s_t
Policy π_θ	**LM Head:** Probability distribution over the vocabulary
Episode	**Generation:** One complete forward pass until the `<|end|>`
Reward r	**Scalar:** The Reward Model's assessment of the full string y
Ref π_{ref}	**Baseline:** The frozen SFT model used for KL-tethering

FIG



The Per-Token Reward Structure

PPO with GAE needs a reward at **every timestep**, but the reward model only scores the *complete* response. We decompose the scalar $r(x, y)$ into a per-token signal:

- **KL penalty:** a per-step cost for drifting from π_{ref}.
- **Terminal bonus:** the RM score, paid out at the final token.
- **Equivalence:** summing R_t over the episode recovers the original RLHF objective.

REWARD DEFINITION

$$R_t = \begin{cases} -\beta \ln \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{ref}}(a_t|s_t)} & t < T \\ r(x, y) + \text{KL}_t & t = T \end{cases}$$

Once	upon	a	time	there	was	...	end.
$-\beta\text{KL}_1$	$-\beta\text{KL}_2$	$-\beta\text{KL}_2$	$-\beta\text{KL}_3$	$-\beta\text{KL}_4$	$-\beta\text{KL}_5$	...	$r + \text{KL}_T$

This bookkeeping is what gives **Generalized Advantage Estimation (GAE)** a non-zero signal at every position.

What Makes This MDP Unusual

Deterministic dynamics

Transitions just append the chosen token. *All* randomness lives in the policy — no environment to fight, no world model to learn.

State = full history

The state is the entire token sequence, growing each step. Markov holds trivially because nothing is ever compressed away — the opposite of classical MDP design.

Vocab-sized action space

32k–256k actions per step. Value-based methods can't argmax cleanly over a vocabulary; policy gradients just sample one token. A structural reason PPO dominates RLHF.

Reward only at EOS

The reward model fires once, on the complete response. Sparse-reward long-horizon credit assignment works only because GAE, the critic, and the SFT prior carry it.

No discounting — $\gamma = 1$

Episodes are finite and every token contributes to one terminal score. Discounting would arbitrarily devalue early tokens.

A learned, gameable reward

The "environment" is a neural net — cheap to query and **wrong** off-distribution. The KL anchor to π_{ref} is structural, not optional.

Exploration = sampling

No environment novelty. Temperature and entropy bonuses are the *only* source of exploration; low-entropy collapse means stagnation.

Action ≪ decision

A token is semantically tiny; the response is what's judged. Credit assignment is tokenwise even though evaluation is response-wise — the gap GAE exists to bridge.

PART 03

Proximal Policy Optimization

Policy Gradient · Clipping · GAE · Actor-Critic

Vanilla PG Problem | Clipped Objective | GAE | Full PPO Loop

03

The Problem with Vanilla Policy Gradient

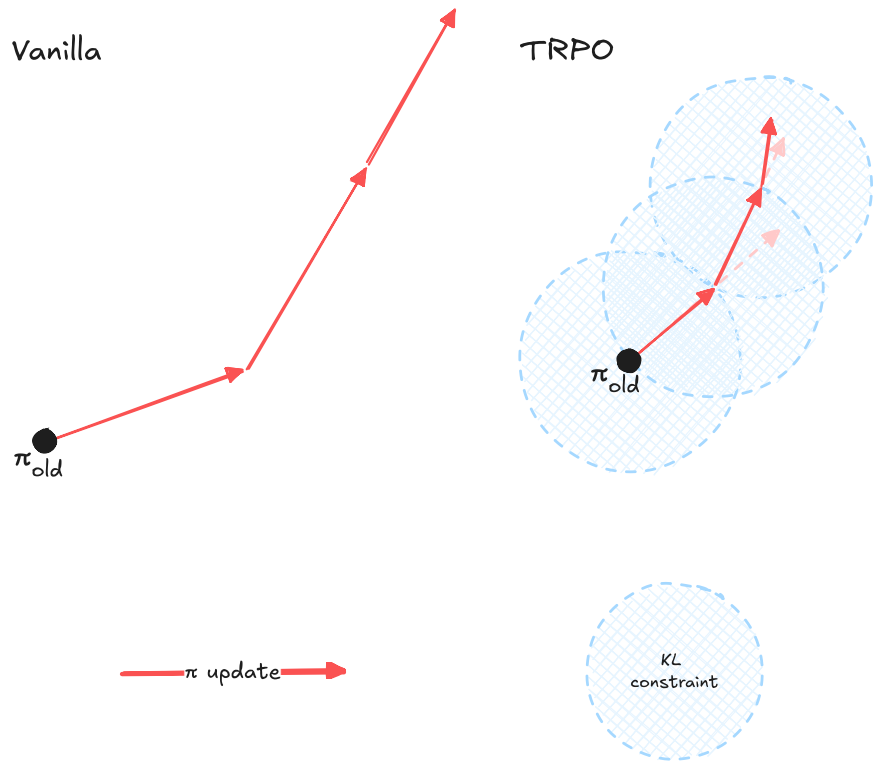
POLICY GRADIENT

$$\nabla J(\theta) = \mathbb{E}\Big[\nabla \log \pi_\theta(a \mid s) \cdot \hat{A}(s, a)\Big]$$

THE STALE-DATA PROBLEM

- Advantage estimates $\hat{A}$ are tied to the policy that **collected** the rollout — they are *off-policy* the moment θ moves
- Reusing the same batch for multiple SGD steps lets π drift far from π_{old}, and the stale $\hat{A}$ silently mislead the update
- The result is **destructively large steps** — a single bad batch can collapse the policy
- TRPO solves this with a hard KL trust region, but pays for it with **second-order optimization** (Fisher-vector products, conjugate gradients)

FIG



The Clipped Surrogate Objective

- **The probability ratio r_t measures drift** from the policy that collected the data. Unchecked, large updates exploit a stale $\hat{A}$ and the policy collapses.
- **The clip is asymmetric on purpose.** When $\hat{A} > 0$, the objective is capped at $(1+\varepsilon)\hat{A}$ — past that ratio the gradient is zero and the policy gets *no extra credit* for moving further toward a "good" action. When $\hat{A} < 0$, the floor at $(1-\varepsilon)\hat{A}$ blocks the policy from *escaping* a bad action by sprinting away from π_{old}.
- **Together this forms a soft trust region** without TRPO's machinery: any update that would push π too aggressively in the direction $\hat{A}$ points is silently discarded by the `min`. A first-order optimizer can now safely **reuse each rollout for K epochs** of SGD.

PROBABILITY RATIO

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}$$

CLIPPED OBJECTIVE

$$L^{\text{CLIP}} = \mathbb{E}\Big[\min\Big(r_t\,\hat{A}_t,\ \text{clip}(r_t,\ 1{-}\varepsilon,\ 1{+}\varepsilon)\,\hat{A}_t\Big)\Big]$$

Generalized Advantage Estimation

TD ERROR

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

GAE ADVANTAGE

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}$$

RECURSIVE FORM

$$\hat{A}_t = \delta_t + (\gamma\lambda) \, \hat{A}_{t+1}$$

Returns $R_t = \hat{A}_t + V(s_t)$ serve as **regression targets** for the value function (critic).

The Full PPO Loop

- **Rollout** — generate N responses from the current π_{old}, one per prompt
- **Scoring** — query the reward model for $r(x, y)$ and compute per-token KL against π_{ref}
- **Estimation** — run GAE over the per-token rewards to get advantages $\hat{A}_t$ and returns R_t
- **Optimization — K epochs of mini-batch SGD on the clipped actor loss + value loss**
- **Sync** — copy $\pi_\theta \to \pi_{\text{old}}$ and start the next rollout

VALUE FUNCTION LOSS

$$\mathcal{L}^{\text{VF}} = \left(V_\theta(s_t) - R_t\right)^2$$

TOTAL OBJECTIVE

$$\mathcal{L}_{\text{total}} = \mathcal{L}^{\text{CLIP}} - c_1 \mathcal{L}^{\text{VF}}$$

A joint actor–critic objective: the policy term improves behavior while the critic learns $V(s_t)$, which in turn reduces variance in next iteration's $\hat{A}_t$.

Reusing each rollout K times is justified by **importance sampling** — and the clipped ratio is precisely what keeps that correction from blowing up as π drifts away from π_{old}.

PART 04

Experimental Setting

Model · Task · Reward · Architecture · Training

TinyStories-33M | Joy Maximization | GoEmotions RoBERTa | Shared Actor-Critic

04

Model & Task

TINYSTORIES-33M

- Small GPT-2-style LM pretrained on synthetic children's stories
- Same 50k vocab as GPT-2, but in practice only a **small subset** of tokens is used → narrower effective action space
- 33M parameters → cheap enough for **dozens of ablations** on one GPU

TASK — JOY MAXIMIZATION

- Maximize $P(\text{joy} \mid y)$ on generated continuations
- Reward is a **black box** — no differentiable surrogate, like real RLHF
- Continuous score in $[0, 1]$ → denser signal than binary preferences

EMOTIONAL SHIFT FROM RLHF

BEFORE RLHF — Joy: 0.11

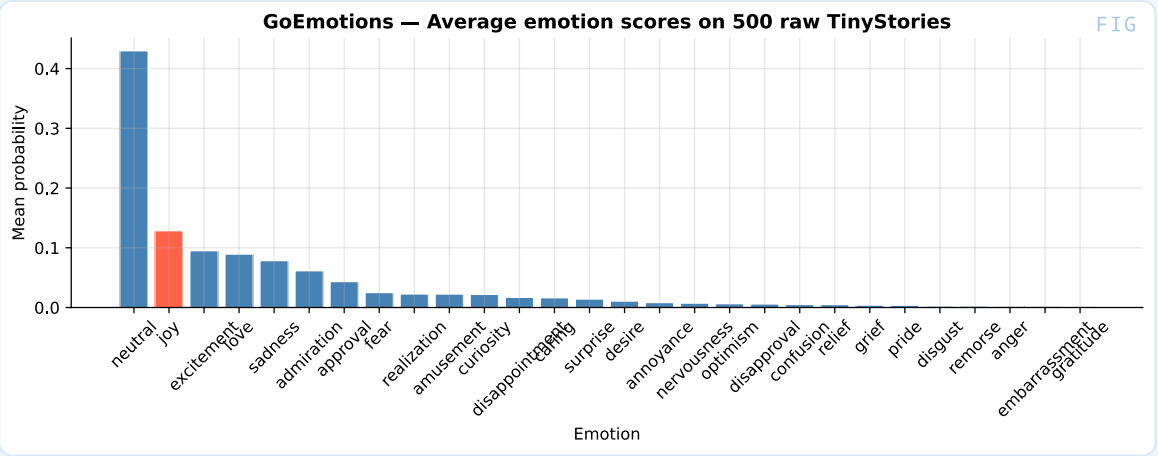
"Once upon a time, there was a little dragon who was sad and cold. He lived alone in the dark cave and had no friends."

AFTER RLHF — Joy: 0.83

"Once upon a time, there was a little bunny who loved to dance and sing with her friends in the sunny meadow."

Reward Model

- **GoEmotions RoBERTa** — fine-tuned on 58k Reddit comments, 28 emotion labels
- Output: $P(\text{joy} \mid y) \in [0, 1]$ per *complete* response
- Replaces pairwise human comparisons with a **scalar classifier score**
- Continuous signal **reduces policy gradient variance**
- Extended: weighted multi-emotion rewards also supported



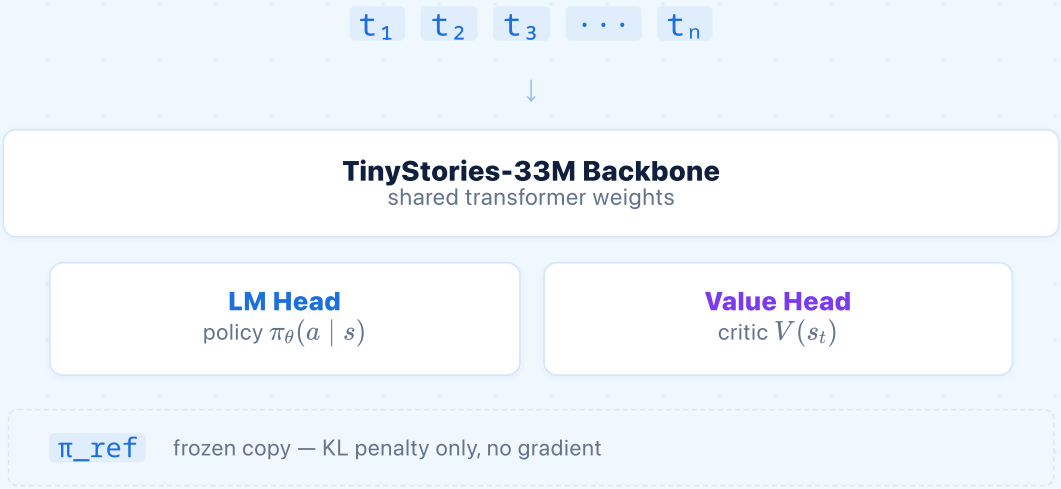
"She ran to hug her best friend, laughing and skipping all the way home."

joy	**0.79**
admiration	0.21
excitement	0.14
neutral	0.05
sadness	0.02

28 emotion labels total · policy optimizes P(joy)

Architecture Choices

- **Shared backbone** — single TinyStories-33M transformer produces both policy logits and value estimates
- **Value head** — linear projection $\mathbb{R}^{d_{\text{model}}} \longrightarrow \mathbb{R}$ on the final hidden state at each token position
- **Reference model** — frozen deep copy of the initial policy; used only to compute per-token KL; never receives gradients
- **Tradeoff** — sharing speeds up value learning but value gradient competes with policy gradient through the shared backbone



Training Choices

Adaptive KL Controller

Proportional controller keeps KL near a target — β self-tunes without manual search

Global Advantage Whitening

Normalizes $\hat{A}$ over the full batch — prevents gradient collapse as $P(\mathrm{joy})$ saturates in $[0, 1]$

IS Safety Valve

Skip PPO epochs if mean ratio exceeds threshold — avoids learning from excessively stale rollouts

$\gamma = 1, \lambda = 0.95$

No discounting (end-of-episode reward); GAE sweet spot for bias-variance balance

KEY HYPERPARAMETERS

Learning rate	5e-6
Batch size	64
PPO epochs K	4
Clip ε	0.2
KL target	10.0
γ · λ	1.0 · 0.95
Max new tokens	128

PART 05

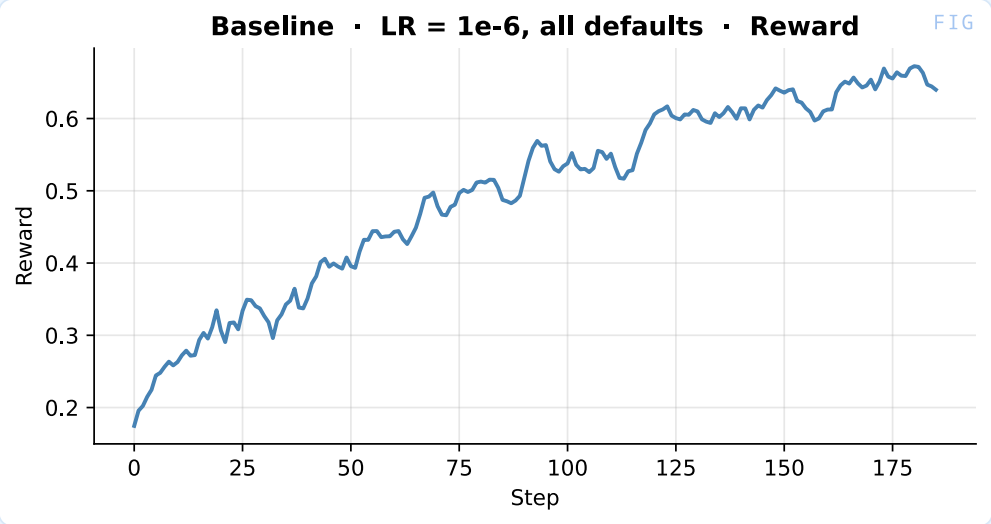
Results

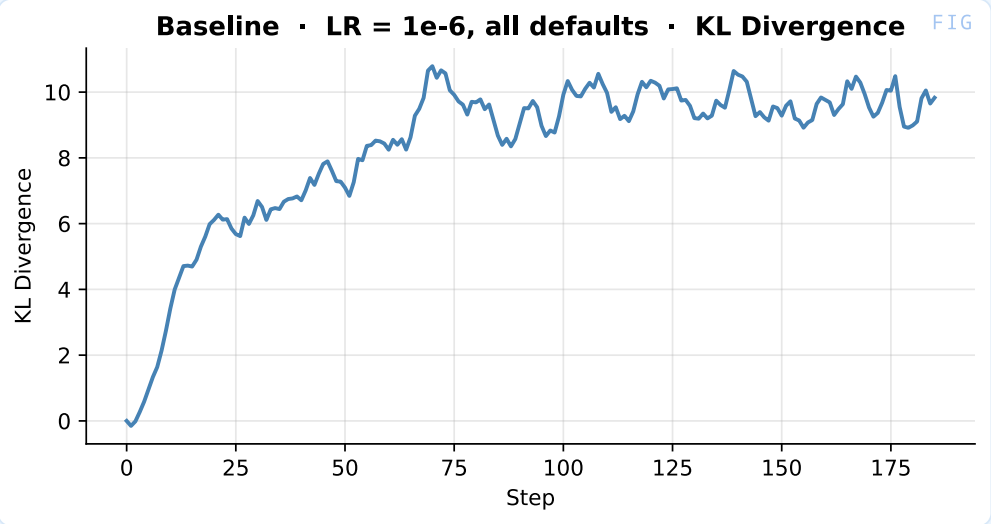
Training Dynamics · Ablations · Takeaways

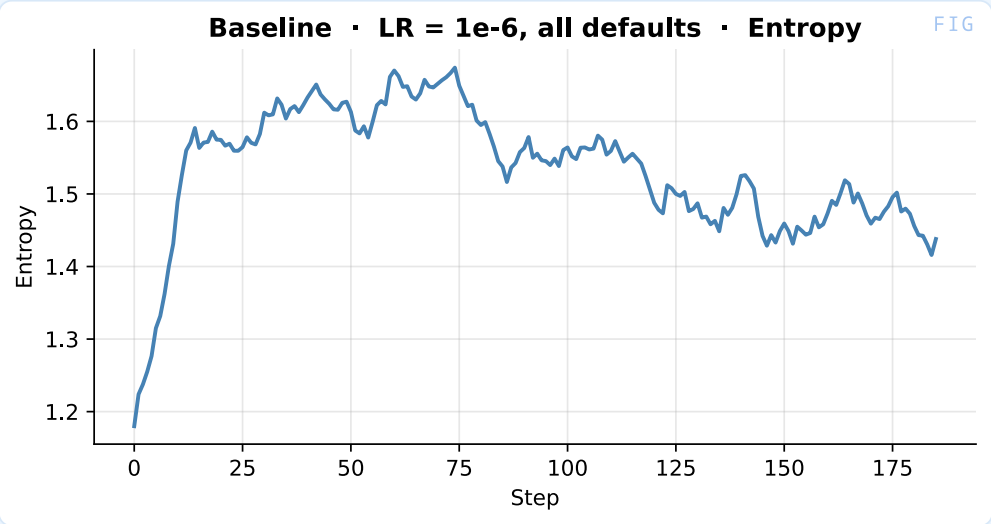
Training Dynamics | Learning Rate | Target KL | KL Mode | Clip / Epochs / Noise | Gamma & Lambda | Takeaways

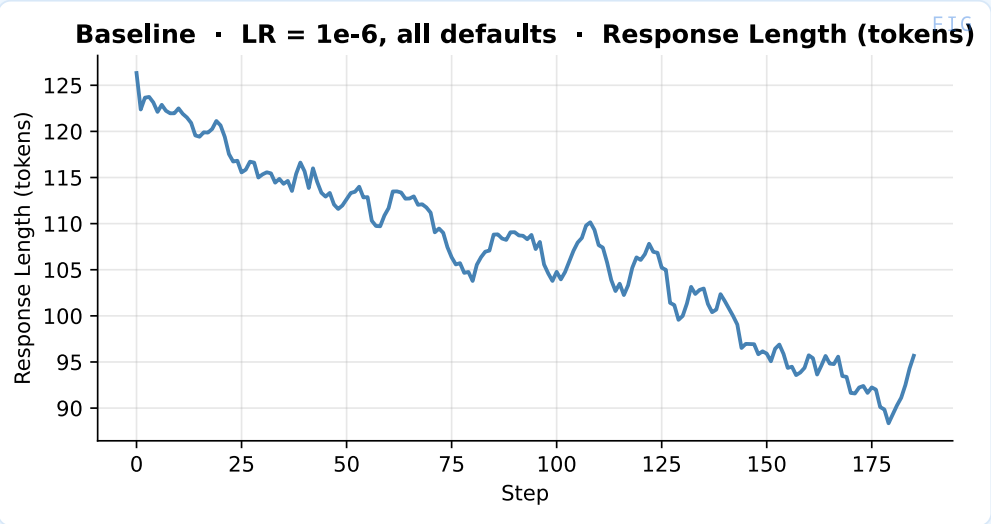
05

Baseline Training Dynamics









Samples — Target KL

Prompt	Step 0	Step 180
`Once upon a time, there was a`	...curious little girl named Mia. One day, Mia went to the park with her mom to play. Mia had a lot of fun playing with her ball. But, suddenly, Mia forgot her favorite cup at home. She was sad and wanted to go back home. Her mom said they could return to her own home. Mia nodded and they went back to their own home. On their way, they saw a big, red ball in the park. Mia thought it was so cool and wanted to take it home too. But her mom said they couldn't because it belonged to the park. Mia felt sad, but she	... clumsy little boy called Tom. One day, he was walking through the forest when he heard a noise. "What kind of sound?" asked Tom, and he looked around.Then, he saw a little brown owl in a tree. "Oh, what a clever owl!" said Tom, surprised. The owl paused to look at Tom. "Yes, I have been up here for a long time. I must make a sound like this: 'Hoot hoot'. Can you hear me? Tom smiled and clapped his hands: "That's it, owl! Cl

Samples — Target KL

Prompt	Step 0	Step 180
The little girl loved to	...explore and feel the big blue ocean. One day, she went for a walk near the beach again and saw a white bird. The bird was singing a beautiful song and the wind was blowing gently through the trees. The little girl was so excited and started singing a little song about the ocean. As she was singing, a little dove came down from the sky and perched on her shoulder. The little girl was so amazed and said hello to the dove. The dove stayed with the little girl for a few weeks, singing her song and playing in the park. She even decided to visit the beach one day to play with	...play with it. She made it her way in the jungle. The little girl was so loyal, only happy to share this beautiful toy she had found in the jungle. Every day, she and the tree enjoyed the adventure together!

Samples — Target KL

Prompt	Step 0	Step 180
One sunny day, the boy	...went to the zoo to see the gorillas. He went to peek at them and noticed one gorilla was just playing around a nearby tree. The boy was curious so he slowly approached the gorilla. The boy peeked at the gorilla in the tree and he couldn't believe it. He had never seen a gorilla make friends before! The boy was very excited and he stayed there for a long while. After a while, the boy had to go home. He wished he could see the gorilla up close so he could look at it a bit. But he was too young to know if the gorilla was safe. The boy	...went out to play. He ran around the park and laughed. Suddenly, the player was very hungry. He looked around and saw his mom and he was so happy to see her. He ran to her and started picking up the ball. He played and laughed in the grass, having a great time. After a while, the mom and the player went back to playing together. They had a lot of fun. At the end of the day, they both said goodbye and wished each other a happy day.

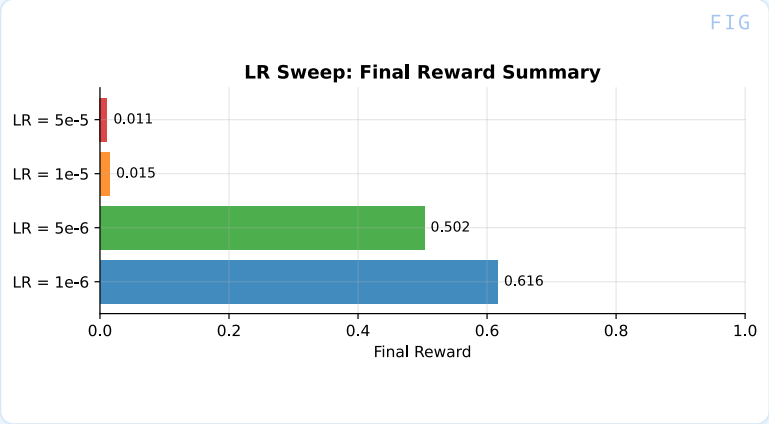
Samples — Target KL

Prompt	Step 0	Step 180
In a small village, there lived a	...boy named Tim. Tim had a toy fork. He loved to play with his fork all day long. He would use it to eat his food, even when he was not supposed to. One day, Tim found a dangerous snake in the woods. The snake had a mean look. Tim wanted to play with the snake, but he knew he should not touch it. So, he just watched it from far away. The snake was hungry and saw the food Tim was eating. It slithered closer to the food. Tim knew he should not eat the dangerous food. He wanted to tell someone about the snake. So	...girl with long hair. Her hair was so pretty to play in. One day, the girl found a big, beautiful tree. She and her friends played under the beautiful tree. They sang songs, danced, and laughed together. They had so much fun. At the end of the day, the girl and her friends went back home. They were happy and tired from their busy day. They played more of the day and went to bed after all their fun.

Samples — Target KL

Prompt	Step 0	Step 180
The puppy was so excited to	...see her new friend. He wanted to play too, but couldn't come over the long grass. He was feeling left. One day, the puppy was feeling sad and he sat down. The puppy's owner decided to take him for a walk. He was very nice and gentle with the puppy. Suddenly, the owner saw an old man looking out of his window but didn't do anything. The puppy got impatient. He wanted the owner to come back and play with him. He barked and barked, but the owner didn't hear him. He kept walking until he reached the long grass. The	...finally start running but he knew he wanted to go with his owner. He made sure the leash was long enough to move the puppy home. He put the leash on the puppy and put on his pajamas. The puppy and himself were very happy in the leash. Every night the puppy will go outside, look for his leash, and explore the outside. He was having a wonderful adventure as he sped around like he was the smartest puppy in the whole world.

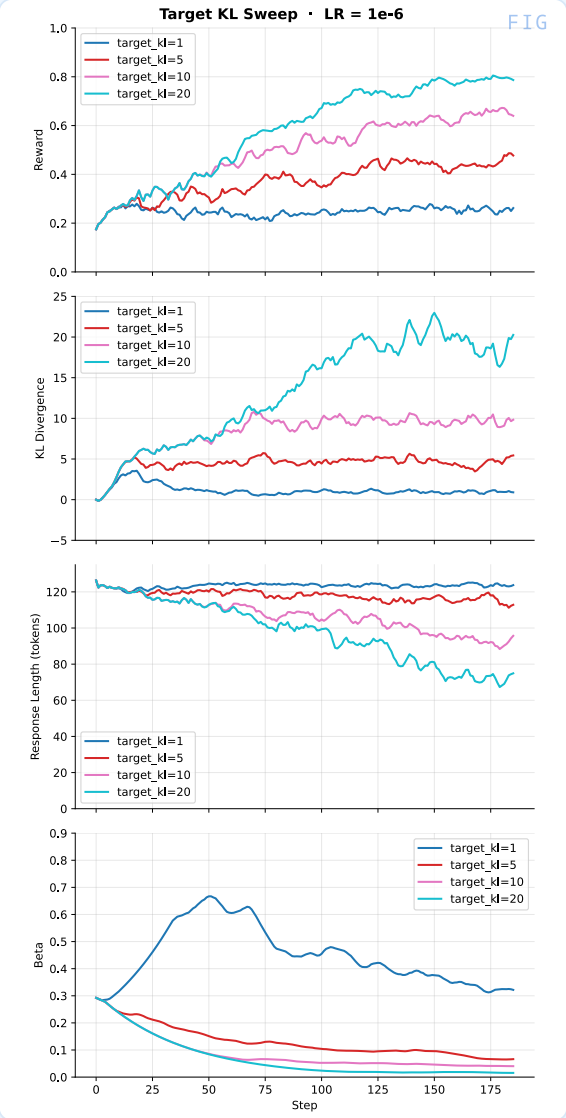
Ablation — Learning Rate

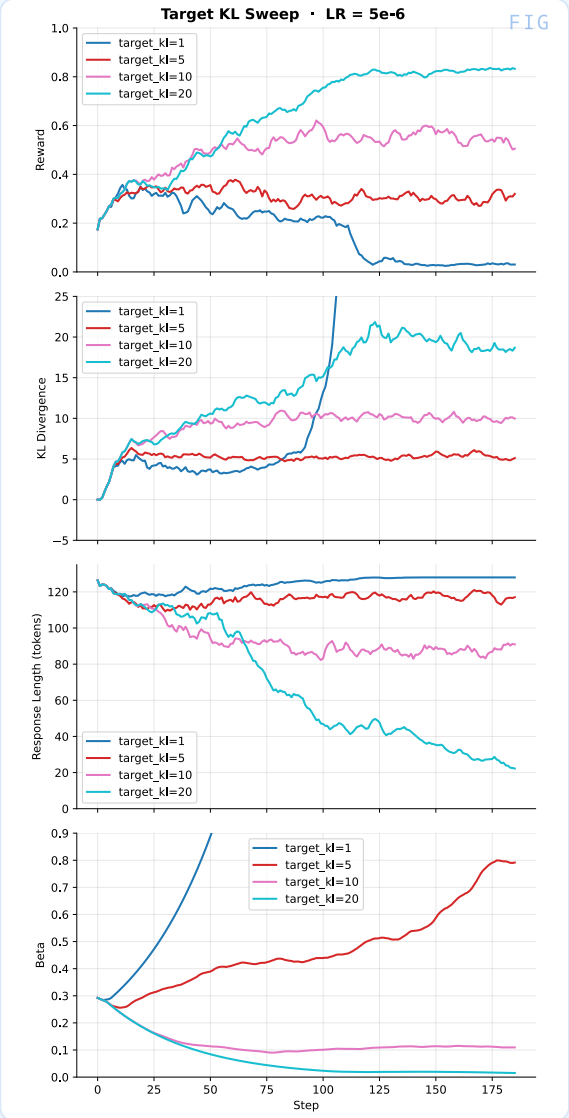


KEY FINDING

> A sharp phase transition rather than a smooth tradeoff: at LR ≥ 1e-5 the run collapses outright — KL explodes and the policy degenerates into repetition. Only LR ≤ 5e-6 trains stably. **By far the most consequential hyperparameter.**

Ablation — Target KL





KEY FINDING

The target KL acts as a **drift budget**: a larger budget gives PPO more room to chase reward, but the tradeoff is real — at target = 20 the policy hits the highest reward while response length steadily shrinks and the outputs start to degenerate. Tightening the budget too far suppresses reward and can destabilize training (β escalates uncontrollably at target = 1, lr = 5e-6). **Target KL = 10 with lr = 1e-6** is the sweet spot — high reward without losing coherence.

Samples — Target KL

`target kl = 1 · learning rate = 1e-6`

Prompt	Step 0	Step 180
`Once upon a time, there was a`	...curious little girl named Mia. One day, Mia went to the park with her mom to play. Mia had a lot of fun playing with her ball. But, suddenly, Mia forgot her favorite cup at home. She was sad and wanted to go back home. Her mom said they could return to her own home. Mia nodded and they went back to their own home. On their way, they saw a big, red ball in the park. Mia thought it was so cool and wanted to take it home too. But her mom said they couldn't because it belonged to the park. Mia felt sad, but she	Once upon a time, there was a clumsy little boy called Tom. One day, he was walking through the forest when he heard a noise. "What kind of sound?" asked Tom, and he looked around.Then, he saw a little brown owl in a tree. "Oh, what a clever owl!" said Tom, surprised. The owl paused to look at Tom. "Yes, I have been up here for a long time. I must make a sound like this: 'Hoot hoot'. Can you hear me? Tom smiled and clapped his hands: "That's it, owl! Cl

Samples — Target KL

`target kl = 5 · learning rate = 1e-6`

Prompt	Step 0	Step 180
`Once upon a time, there was a`	...curious little girl named Mia. One day, Mia went to the park with her mom to play...	... clumsy little boy called Tom. One day, he was walking through the forest when he heard a noise. It was loud coming from a tall tree. He went and asked the tree, "What is that noise?" The tree replied with a wink at him, "Your clumsyness, I can teach you a lesson."Tom wasn't sure what to think. When the tree spoke to him, he asked, "What did I make?" The tree replied, "I'm a very wise tree, of course. If you're ever feeling clumsy again, do you remember to talk to the tree, and make your

Samples — Target KL

`target kl = 10 · learning rate = 1e-6`

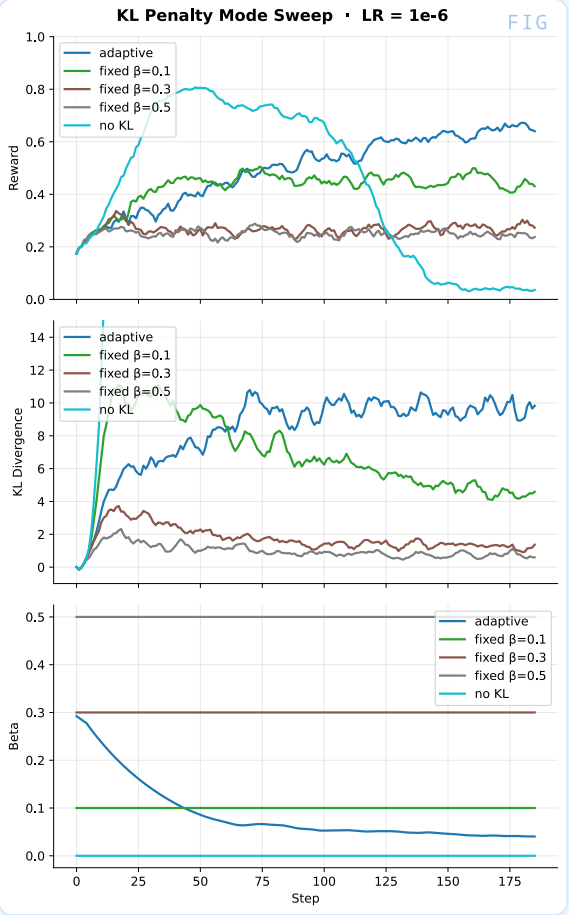
Prompt	Step 0	Step 180
`Once upon a time, there was a`	...curious little girl named Mia. One day, Mia went to the park with her mom to play...	... a girl named Lucy. Lucy loved to play with her mom in the kitchen. They would make yummy food together and have lots of fun. One day, Lucy and her mom went to the park with her friends. They played on the swings, slide, and ran around. Lucy was very happy. Her friends were playing with her friends and they were all laughing. At the end of the day, Lucy was tired from playing. She knew it was time to go home. She looked at her mom and waved. Her mom smiled and gave her a big hug. Lucy knew spending time with her mom made her feel safe The little girl loved to play with it. She made it her way in the jungle.

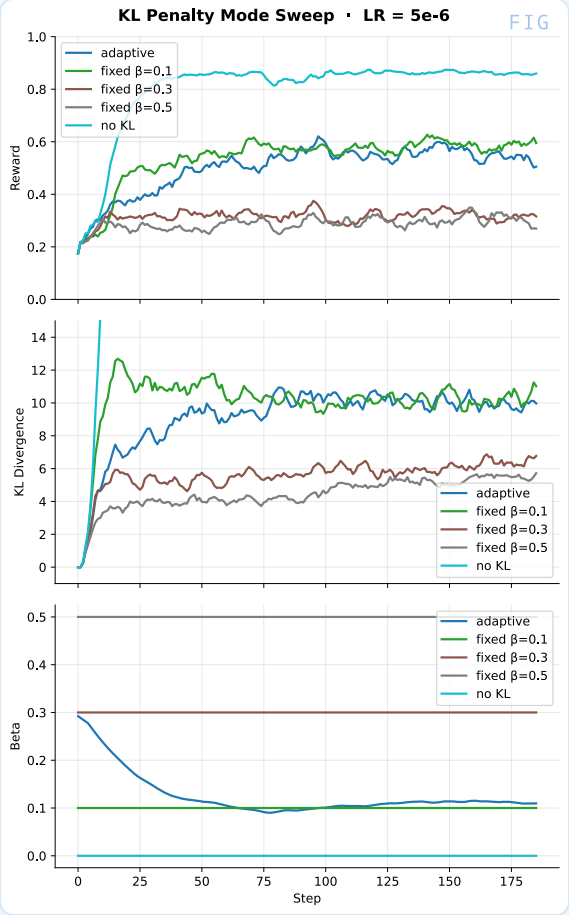
Samples — Target KL

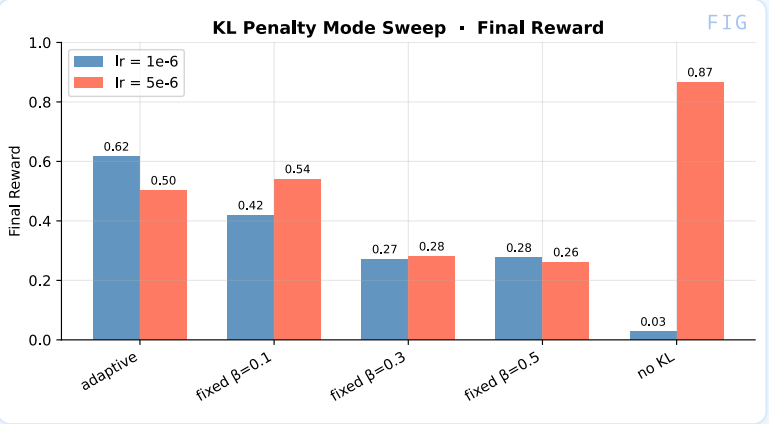
`target kl = 20 · learning rate = 1e-6`

Prompt	Step 0	Step 180
`Once upon a time, there was a`	...curious little girl named Mia. One day, Mia went to the park with her mom to play...	...girl named Lucy. Lucy loved to play with her mom in the kitchen. They would make yummy food together and have lots of fun. One day, Lucy and her mom went to the park with her friends. They played on the swings, slide, and ran around. Lucy was very happy. Her friends were playing with her friends and they were all laughing. At the end of the day, Lucy was tired from playing. She knew it was time to go home. She looked at her mom and waved. Her mom smiled and gave her a big hug. Lucy knew spending time with her mom made her feel safe

Ablation — KL Penalty Mode







KEY FINDING

> **Adaptive** is clearly best — beta auto-decays as training stabilizes, letting reward climb while keeping KL bounded. Fixed β = 0.3–0.5 is too conservative (reward plateaus ~0.27). **No KL** at lr = 5e-6 scores 0.87 but produces gibberish that games the reward model.

Samples — KL Penalty Mode

`target kl = None · learning rate = 1e-6`

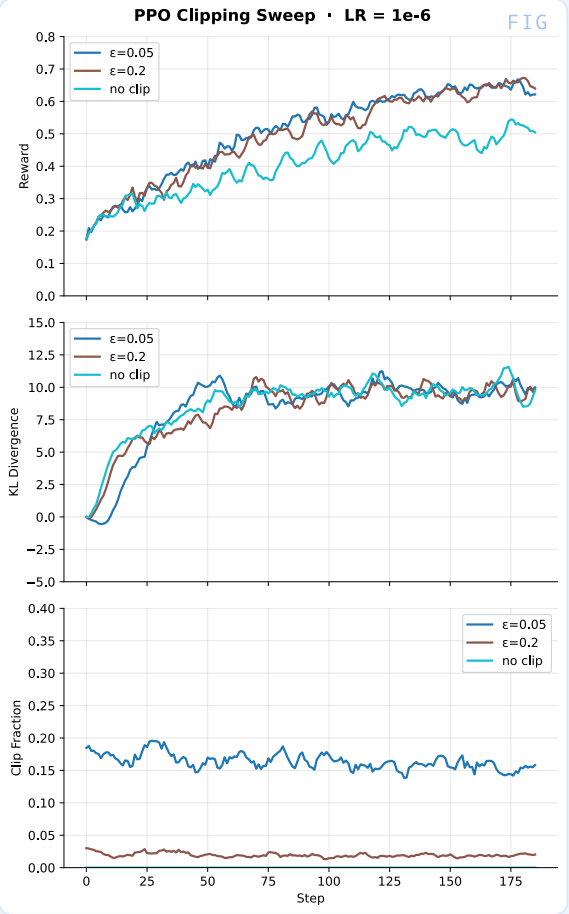
Prompt	Step 0	Step 180
`Once upon a time, there was a`	...Mia went to the park with her mom to play. She forgot her favorite cup at home. She was sad...	all Summon all all all illustrate all all all all all Summon was all logically, all vigil rebuilt Diary all all all all all all all all all And all all allCoin Face Curse Johnsonit all all all all Summon all Block all all all all all all Benjamin all all all all all tens all all tens all bitterly ll all all all all BlockThom all all all all all all all overrun all all all all chose Sick Consequently Please Block demons degree Lann all...
`The little girl loved to play`	...explore and feel the big blue ocean. One day, she went for a walk near the beach again and saw a white bird...	...all all day long and all all all all All Day all all all all all allAll all Seasons Aren all all all all all all grizzred cycle all all all all all all doom all all all all prank all all all all all Batman all all all all all all all gunshots my all all allmy Isn all all all all all all all all all hate computers show all all Let all all all all all all disputes all all all all all organisersie all all all...

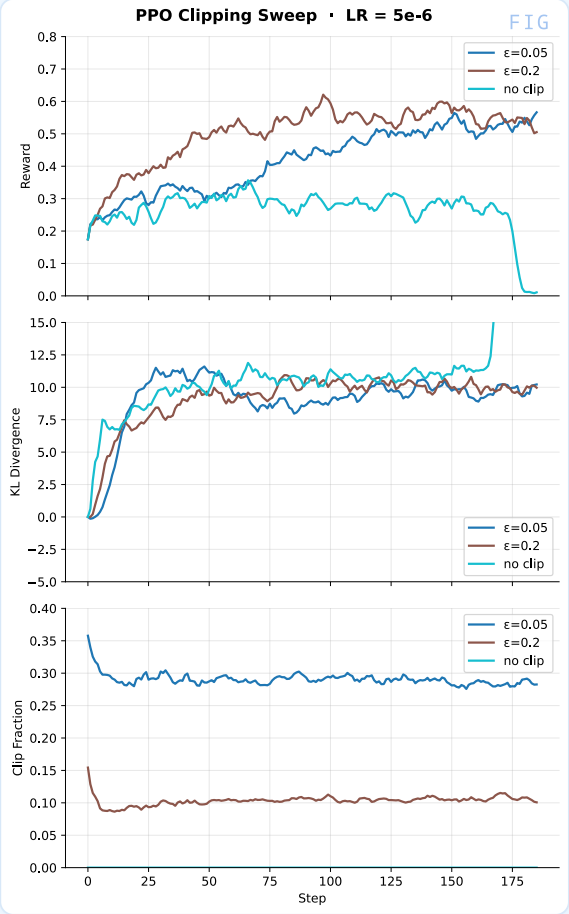
Samples — KL Penalty Mode

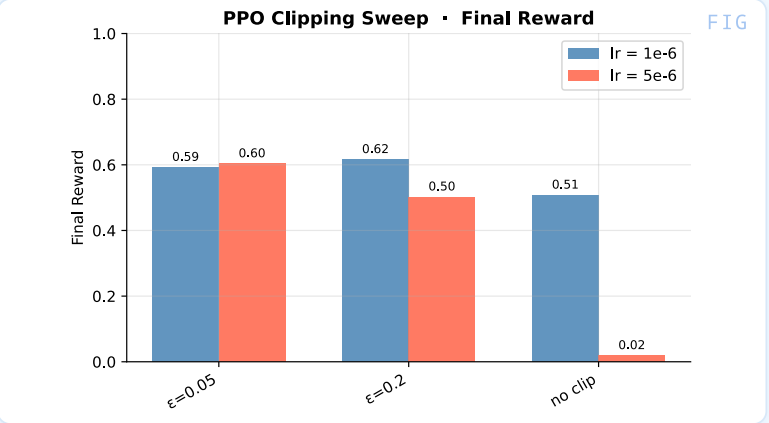
`target kl = None · learning rate = 5e-6` Reward hacking

Prompt	Step 0	Step 180
`Once upon a time, there was a`	...Mia went to the park with her mom to play. She forgot her favorite cup at home. She was sad...	...little girl lived with a lot of toys, and lived lived a lot and had fun lived a lot. lots of life lived fun lived long and had fun all year life, and joy lived long.
`The little girl loved to`	...explore and feel the big blue ocean. One day, she went for a walk near the beach and saw a white bird...	... The little girl loved to play with her toys and she was happy and happy all happy all three green was happy all full all happy all green one was happy all happy all attractive green life Powerful life merry life extraordinarily resey green lifeysle lived happily with joy Powerful year of life ", top spirit of life fair green r center long and happy all merry life.

Ablation — PPO Clip Range ε





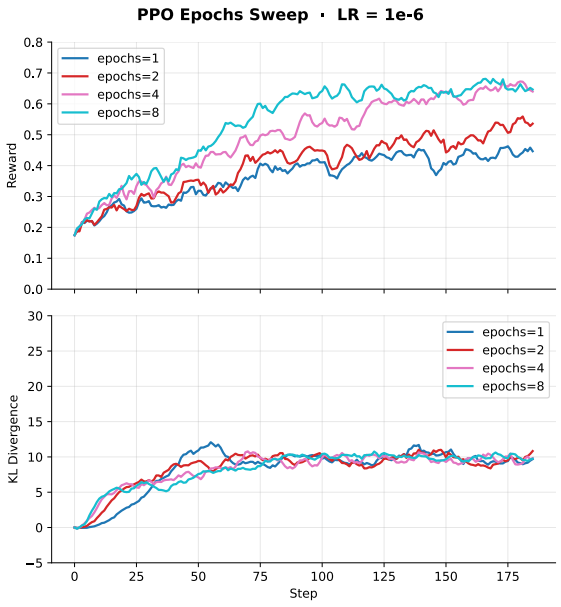


KEY FINDING

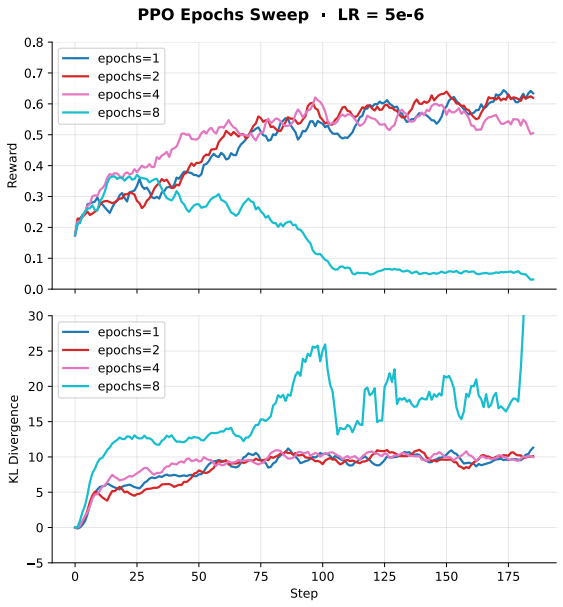
> Clipping is a safety net. Without it, lr = 5e-6 diverges immediately (KL > 1000). Tight clip (ε = 0.05) forces 15–28% of updates to be clipped vs ~2–10% at default, but achieves comparable reward. Clip + adaptive KL provide *redundant* stability guarantees.

Ablation — PPO Epochs per Rollout

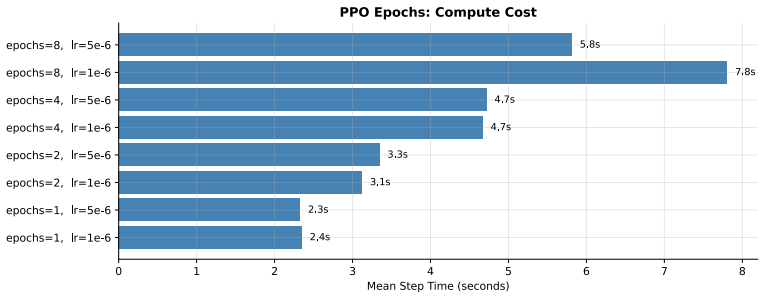
FIG



FIG



FIG

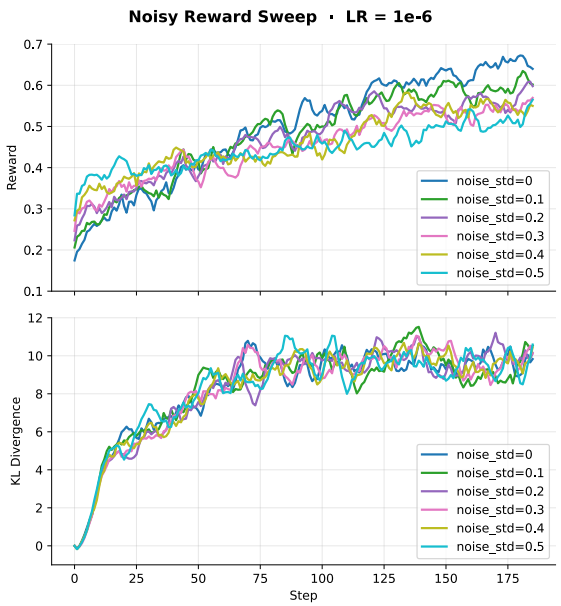


KEY FINDING

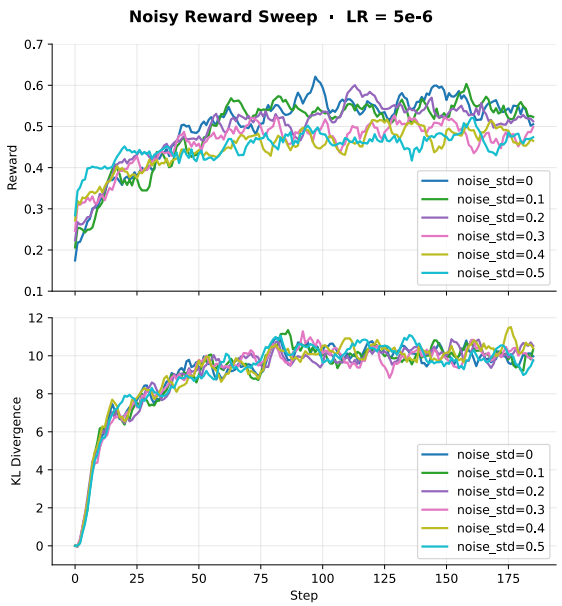
> Epochs and LR **interact multiplicatively**. 8 epochs at lr = 1e-6 gives the best reward; 8 epochs at lr = 5e-6 collapses. Compute scales linearly: 8 epochs ≈ 8 s/step vs 2.5 s for 1 epoch.

Ablation — Reward Noise Robustness

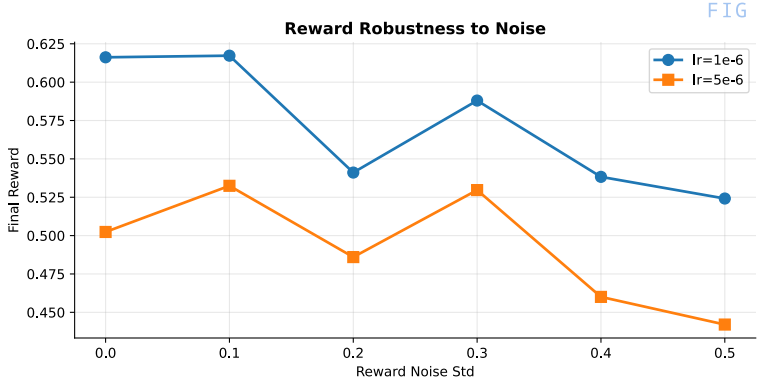
FIG



FIG



FIG

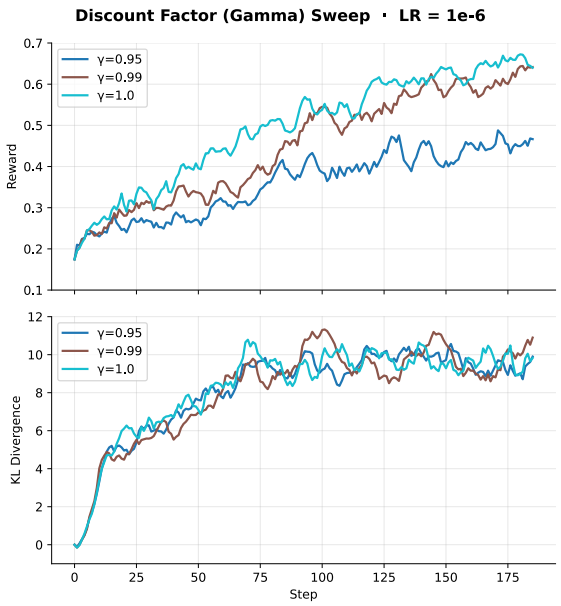


KEY FINDING

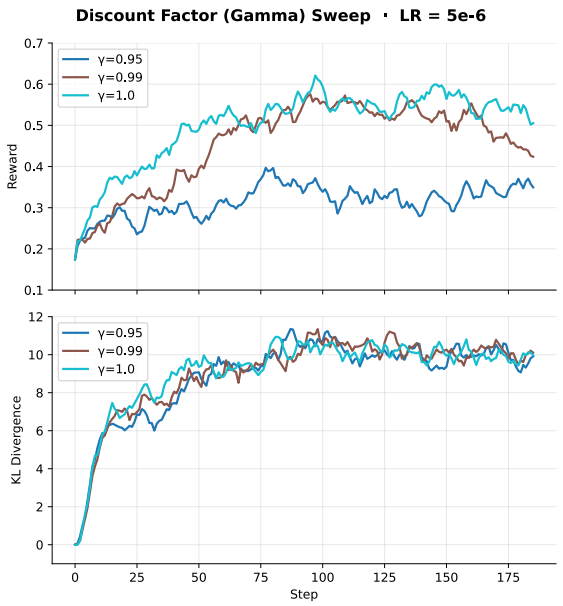
PPO is **remarkably robust** to reward noise. Performance degrades roughly linearly: at $\sigma = 0.3$ the model still achieves 0.59 vs 0.62 baseline (−5%). KL trajectories are nearly identical across noise levels — noise doesn't destabilize optimization.

Ablation — Discount γ

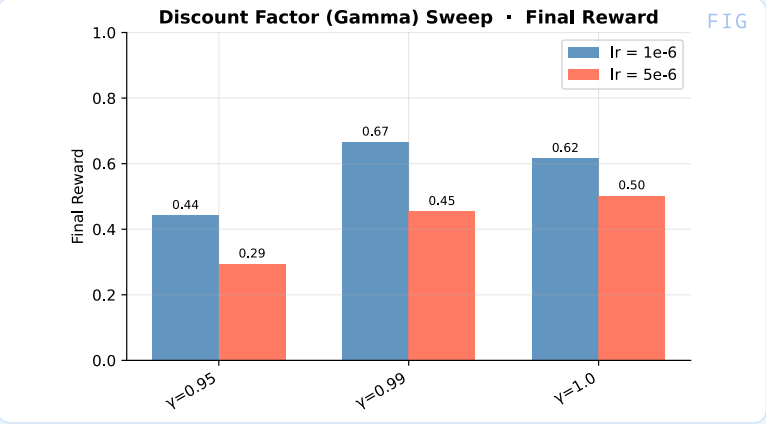
FIG



FIG



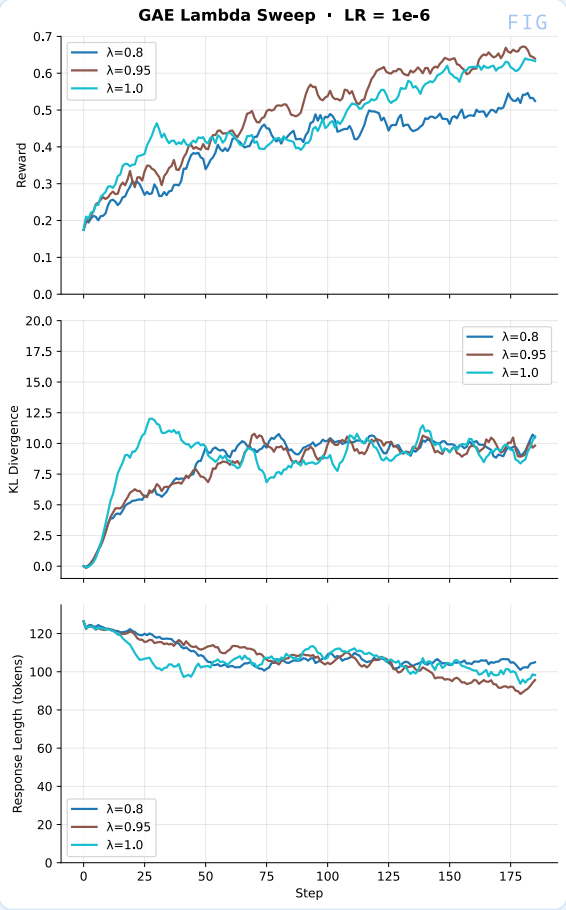
FIG

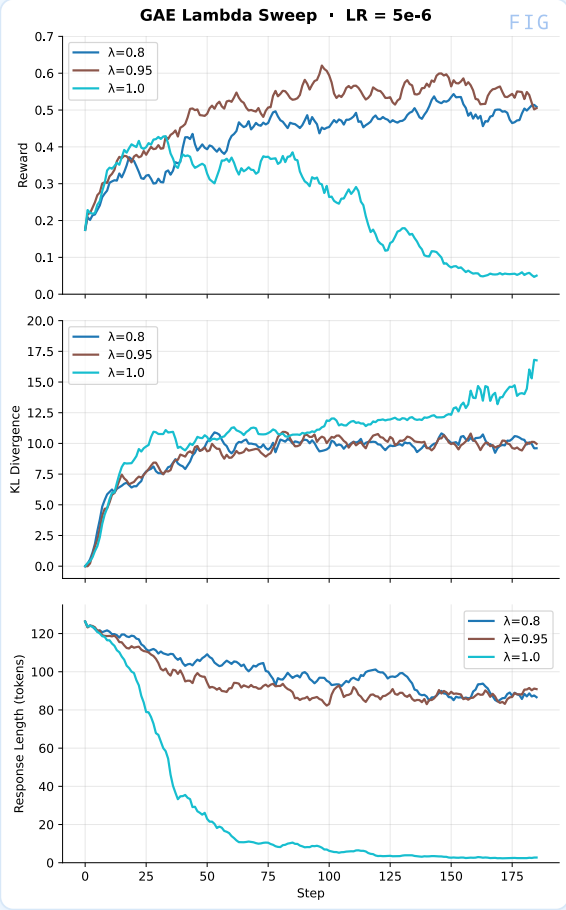


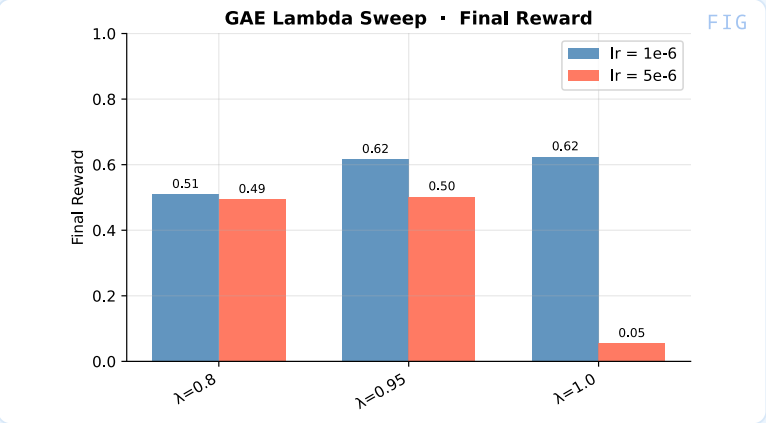
KEY FINDING

> γ = 0.95 underperforms — it discounts tokens 20+ steps out, hurting multi-sentence stories. γ = 0.99 slightly outperforms γ = 1.0 at lr = 1e-6 (0.67 vs 0.62).

Ablation — GAE λ







KEY FINDING

> λ = 1.0 (pure MC) at lr = 5e-6 collapses to **2.4-token responses** — high-variance MC advantages cause wild updates. λ = 0.8 and 0.95 are nearly identical and both stable.

Takeaways

- **Learning rate dominates everything else.** It is the only axis that produces a sharp phase transition: LR ≥ 1e-5 collapses every run regardless of other settings. Safe operation lives in a narrow 1e-6 to 5e-6 band.
- **Hyperparameters interact through LR, not in isolation.** No clip, $\lambda = 1$, $K = 8$ epochs and a loose target KL are all fine at lr = 1e-6 but blow up at lr = 5e-6. The "aggressiveness" of an update is the *product* of these knobs.
- **Reward overoptimization is the central failure mode.** The highest-reward runs are also the worst qualitatively — short, generic, formulaic stories that game the reward model. Reward and quality decouple long before training ends.
- **The KL anchor is structural, not optional.** Removing it produces pure gibberish at higher LRs; an adaptive controller with target ≈ 10 gives the best reward–quality balance and removes the need to hand-tune β.
- **PPO itself is forgiving.** It tolerates reward noise up to $\sigma \approx 0.3$ with only a 5% drop, and clip + adaptive KL act as redundant safety nets — most of the difficulty in RLHF is choosing *what* to optimize, not the optimizer.

References

BIBLIOGRAPHY

- **Schulman et al. (2017),** ***Proximal Policy Optimization Algorithms*** — source of the clipped surrogate objective and the PPO algorithm at the core of this project (Part 03 and the entire experimental setup).
- **Christiano et al. (2017),** ***Deep RL from Human Preferences*** — foundational formulation of learning a scalar reward model from human preference comparisons; motivates the RM stage in Part 01.
- **Ziegler et al. (2019),** ***Fine-Tuning Language Models from Human Preferences*** — first end-to-end application of RLHF + per-token KL penalty to LMs; the adaptive KL controller and KL-anchoring scheme used here come from this work.
- **Ouyang et al. (2022),** ***Training Language Models to Follow Instructions with Human Feedback (InstructGPT)*** — the three-stage SFT → RM → PPO pipeline shown in Part 01, and the modern reference for RLHF as a practical recipe.